

# Chapter 3: System Development

## 3.1 Requirements and Analysis

### 3.1.1 Problem Definition

It is critical to identify and diagnose brain tumors through MRI scans quickly and correctly in order to determine the most effective treatment options and increase the likelihood of patient recovery. Unfortunately, the traditional approach of manually reviewing these images can be frustrating: It takes a long time to review; the result is highly dependent upon the skillset of the radiologist; and often, because different radiologists interpret MRI images differently, discrepancies between interpretation may lead to misdiagnosis.

Our project eliminates these obstacles by providing a comprehensive deep learning platform that enhances the quality of MRI images, and classifies the detected abnormalities into one of four categories: glioma, meningioma, pituitary, or no tumor. To enhance the overall usability of the platform, we have included a simple web application that allows users to upload their MRI images and receive an analysis instantly.

### 3.1.2 Functional Requirements

The MRI imaging system must be able to manage the entire MRI process, from the beginning (Data Intake and pre-processing) through the end (Model deployment) and any predictions must be delivered so they fit in with the clinical process. The system must combine both targeted improvements to individual slices of the MRI (images) and the use of Deep Learning to classify a Tumor as a means of producing outputs that can withstand review by other professionals, but be made available to the end user/Physician through an easy-to-use app. More specifically our goals will be to provide:

1. An Automated Loading of MRI Images - all images of interest will be loaded automatically and after being cleaned, so that their quality is assured.
2. Research and Compare Separate Enhancement Models like U-Net, DnCNN and EDSR, based on performance against commonly used Standard metrics like PSNR and SSIM.
3. Generate Several different Classifications Models, including: Baseline CNN, VGG16, ResNet50 and Xception (all written in PyTorch), and EfficientNetB4 (in TensorFlow/Keras), and then compare their Relative Accuracy and Efficiency.

4. Produce Key Clinical Measurement Metrics to Evaluate Performance - (accuracy, precision, recall, F1-score and Specificity) segregated by Class; ROC-AUC Scores (per Class and Overall) and Full Confusion matrices (complete).
5. Deploy to clinicians and end users with a user-friendly Gradio interface where they can upload scans in an easy and convenient way, and receive back not only the predicted type of tumor with confidence values but also an easy-to-read report-style summary of the results, specifically designed for real world use.

### 3.1.3 Non-Functional Requirements

1. **Accuracy:** The classification model must achieve a test accuracy of >95% to be clinically relevant.
2. **Inference Latency:** The total processing time (from upload to diagnosis) must be under 2 seconds.
3. **Robustness:** The system must handle variations in scanner quality, zoom, and patient orientation.

### 3.1.4 Datasets Used

The two publicly accessible MRI datasets for brain tumors utilized in this project contain images divided into four categories: Glioma, Meningioma, Pituitary, and No Tumor. The variety of MRI scans available from these two datasets allows for extensive training and testing of the models developed.

The two datasets are:

1. **Figshare Brain Tumor Dataset** - Contains 2D MRI images on PNG and JPG file formats. The images included within this dataset are labelled according to the four tumour types and are frequently used in research to develop algorithms to classify brain tumours.
2. **Brain Tumor MRI Dataset from Kaggle (masoudnickparvar)** - This dataset consists of high-quality T1-weighted contrast-enhanced MRI scans. Due to the high-quality T1-weighted contrast-enhanced MRI scans found within this dataset, it provides clearer and more detailed images than the previous one, which allows for more effective training of models and reduces the amount of ambiguity present during classification.

## 3.2 Project Design and Architecture

### 3.2.1 System Architecture and Workflow

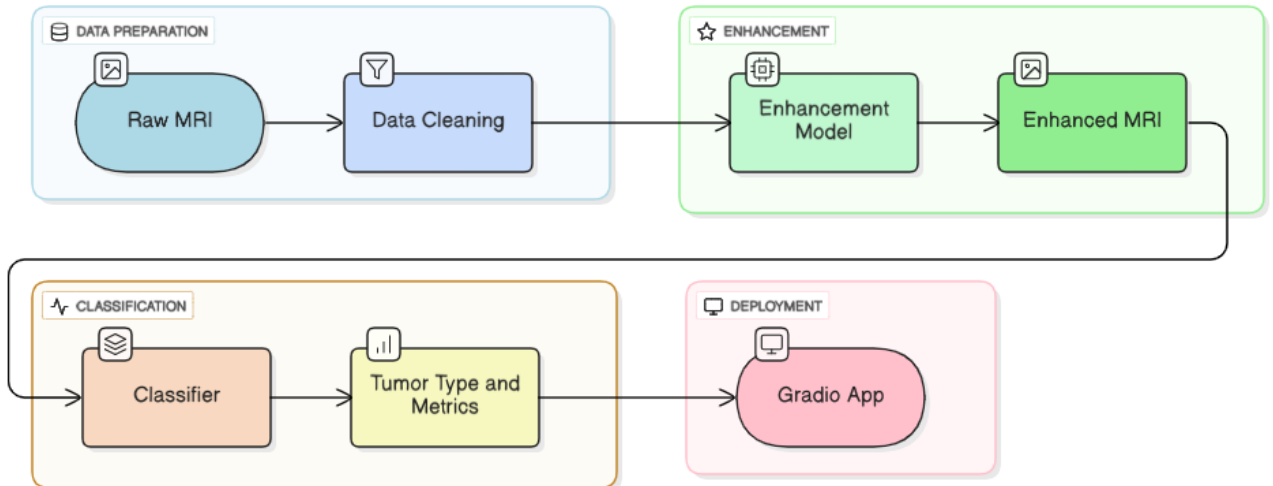


Figure 3.1: Workflow of the project

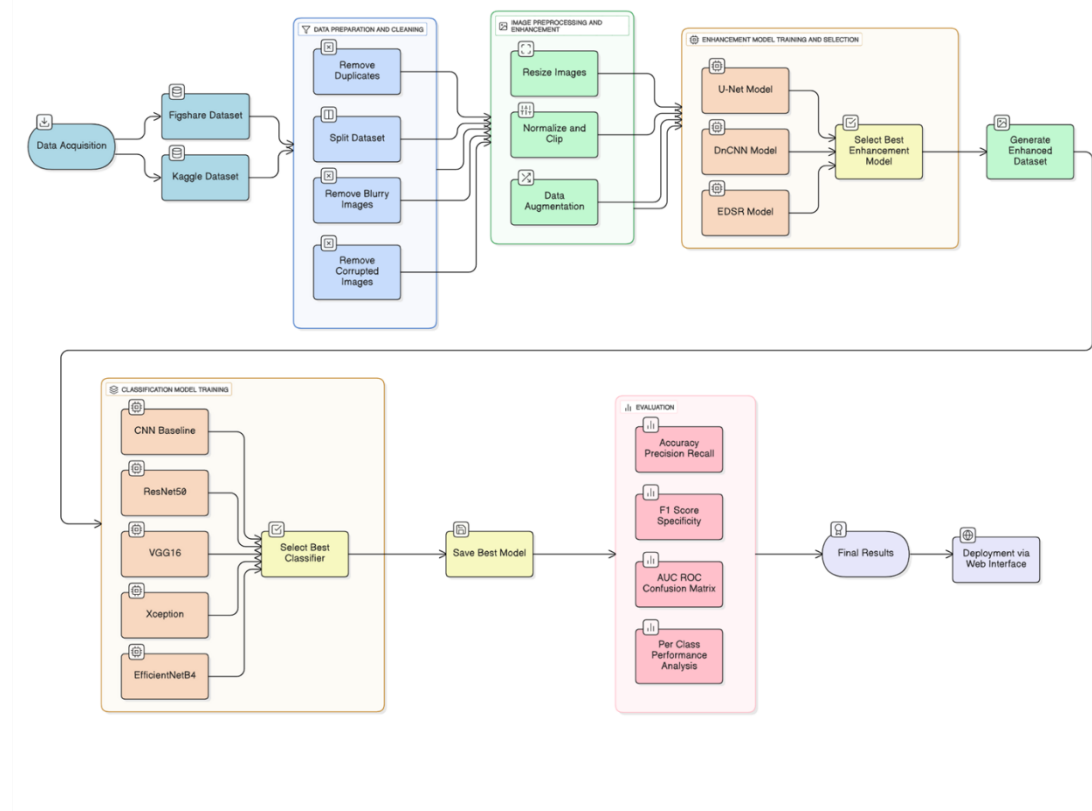


Figure 3.2: Detailed Workflow and Architecture of the Project

#### 1. Raw Data → Cleaning & Preprocessing

- i. Remove corrupted / low-quality images
- ii. Normalize, resize, convert to grayscale, balance classes

- iii. Split into train / validation / test sets.

## 2. Image Enhancement Module (PyTorch)

- i. Train U-Net, DnCNN, EDSR on synthetic noisy MRI pairs.
- ii. Evaluate them on a clean test set using PSNR and SSIM.
- iii. Choose **best enhancement model** (U-Net, based on SSIM).
- iv. Use best model to create an **enhanced dataset**.

## 3. Classification Module – PyTorch

- i. Train Baseline CNN, ResNet50, VGG16, Xception using the **U-Net-enhanced dataset**.
- ii. Compare metrics on a held-out test set.

## 4. Classification Module – TensorFlow (EfficientNetB4)

- i. Separate pipeline with heavy preprocessing and class-weighted training.
- ii. Train EfficientNetB4 on combined datasets (380×380 RGB inputs).
- iii. Evaluate image-level metrics and confusion matrix.

## 5. Deployment Module – Gradio Web App

- i. Load the trained EfficientNetB4 model.
- ii. Accept an uploaded MRI slice.
- iii. Preprocess identically to training.
- iv. Display class probabilities and a styled diagnosis report.

### 3.2.2 Core Convolutional Mathematics

All models in this system rely on the discrete convolution operation.

For an input image  $I$  of size  $H \times W$  and a filter  $K$  of size  $k \times k$ , the output feature map  $S$  is:

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n) \cdot K(m, n) + b$$

**Optimization (Adam):** We utilize the Adam optimizer, which adapts learning rates based on the first moment ( $m_t$ ) and second moment ( $v_t$ ) of the gradients:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

### 3.2.3 Preprocessing Architectures

#### 1. DnCNN (Denoising CNN)

- **Purpose:** Removal of Rician noise from MRI scans.
- **Math (Residual Learning):** Instead of learning the clean image mapping  $x$ , DnCNN learns the residual noise mapping  $R(y) \approx v$ .

$$y = x + v \rightarrow \hat{x} = y - R(y)$$

The loss function minimizes the Frobenius norm between estimated and true noise:

$$L(\Theta) = \frac{1}{2N} \sum_{i=1}^N \|R(y_i; \Theta) - (y_i - x_i)\|_F^2$$

#### 2. EDSR (Enhanced Deep Super-Resolution)

- **Purpose:** Upscaling  $2\times$  low-resolution scans without introducing blur.
- **Math (Sub-Pixel Convolution):** EDSR uses a Pixel Shuffle layer to rearrange channel depth into spatial dimensions.

$$PS(T)_{x,y,c} = T_{\lfloor x/r \rfloor, \lfloor y/r \rfloor, c \cdot r \cdot \lfloor bmod(y,r) + \lfloor bmod(x,r) \rfloor}$$

where  $r$  is the upscaling factor.

#### 3. U-Net (Region of Interest Extraction)

- **Purpose:** Skull stripping to isolate the brain tissue.
- **Math (Skip Connections):** The decoder concatenates feature maps from the encoder ( $x_{enc}$ ) to preserve spatial details:

$$x_{dec} = \text{Concat}(\text{UpConv}(z_l), x_{enc})$$

- **Math (Dice Loss):** Optimized to handle class imbalance between background and tumor:

$$Dice = \frac{2 |X \cap Y|}{|X| + |Y|}$$

### 3.2.4 Classification Architectures

#### 4. Custom CNN (Baseline)

- **Design:** A sequential stack of 4 Convolutional blocks followed by Dense layers.
- **Math (ReLU Activation):** Applied to introduce non-linearity:

$$f(x) = \max(0, x)$$

#### 5. VGG16 (Visual Geometry Group)

- **Design:** A deep stack of 13 convolutional layers using small  $3 \times 3$  filters.
- **Math:** The receptive field of two stacked  $3 \times 3$  layers is effectively  $5 \times 5$ , but with fewer parameters and increased non-linearity.

#### 6. ResNet50 (Residual Networks)

- **Purpose:** Solving the vanishing gradient problem in deep networks.
- **Math (Residual Block):** The network learns a residual function  $F(x)$ , resulting in the mapping:

$$H(x) = F(x) + x$$

This allows gradients to flow directly through the identity shortcut  $+x$  during backpropagation.

#### 7. Xception (Extreme Inception)

- **Purpose:** Efficiency through decoupling spatial and channel correlations.
- **Math (Depthwise Separable Convolution):**

$$Conv(K, I) \rightarrow Depthwise(K_{dw}, I) + Pointwise(K_{pw}, I)$$

This reduces the computational cost by a factor of roughly

$$\frac{1}{N} + \frac{1}{k^2}.$$

#### 8. EfficientNetB4 (Champion Model)

- **Purpose:** Optimal scaling of network dimensions.
- **Math (Compound Scaling):** It scales width ( $w$ ), depth ( $d$ ), and resolution ( $r$ ) uniformly using a compound coefficient  $\phi$ :

$$d = \alpha^\phi, w = \beta^\phi, r = \gamma^\phi$$

subject to:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

For B4,  $\phi = 4$ , providing the optimal balance for  $380 \times 380$  (or resized  $224 \times 224$ ) inputs.

### 3.3 Data Preparation

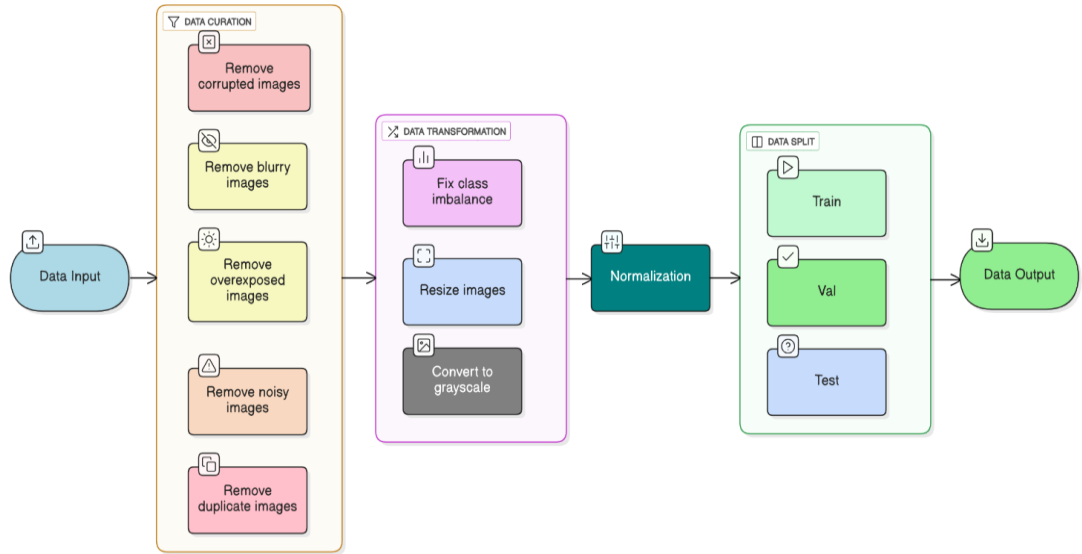


Figure 3.3: Data Preparation

All MRI images were first passed through a structured cleaning pipeline to ensure only high-quality scans were used.

- **Corrupted/unreadable images** were removed.
- **Blurry images** were detected using Laplacian variance and discarded.
- **Overexposed scans** were removed based on histogram analysis (too many pixels near 0 or 255).
- **Highly noisy or artifact-affected images** were eliminated manually or based on noise thresholds.
- **Duplicate images** were identified using perceptual hashing or checksums.
- **Class imbalance** was handled using class-balanced sampling (PyTorch) and class weights (EfficientNetB4).
- Images were resized to:
  - a.  $224 \times 224$  pixels (PyTorch models)
  - b.  $380 \times 380$  pixels (EfficientNetB4 model)
- MRI grayscale scans were converted to 1 channel, then duplicated to 3 channels for pretrained CNNs.
- **Normalization:**

- PyTorch: scaled to [0,1][0,1][0,1] and normalized(mean = 0.5, std = 0.5)
- TensorFlow: used `efficientnet.preprocess_input`
- Dataset splitting:
  - 70% training, 20% validation, 10% testing (PyTorch)
  - Separate train/test directory structure (TensorFlow)
- Masks were saved for future segmentation research.

### 3.3.2 Data Augmentation

We incorporated dynamic augmentation methods into the training process to reduce the likelihood of overfitting and increase the models' ability to generalize. We normalized all pixel intensities by dividing each intensity by 255 to bring all values between 0-1, added a max shear distortion of 0.2, zoomed in/out (80%-120%) and randomly flipped the images horizontally at the time of training. By introducing these variations into each image we were able to replicate factors such as positional change and zooming unique to real-world scenarios. The end result was to create classifiers that perform much better across a wide variety of clinical situations.

## 3.4 Implementation Details

To address the challenges of processing images, we developed three unique neural networks (in PyTorch) to handle the various preprocessing tasks in an easily adaptable manner. DnCNN was chosen for its ability to remove noise, EDSR for enhancing image quality when necessary, and U-Net for skull stripping/extraction of specific areas of interest. Each network has been encapsulated within its own class module, which allows us to easily access the appropriate network based on the needs of the scan at the time of run. TensorFlow version 2.15.0 with an associated Keras 3.0 interface serves as the underlying framework of our processing system, while Tesla T4 GPUs from NVIDIA allow us to efficiently perform the high degree of matrix calculations that these systems require.

| Model         | Architecture                | Learning Algorithm | LR                 | Batch | Dropout | Epoch | Early-Stoppped | Loss Function                      | Activations   |
|---------------|-----------------------------|--------------------|--------------------|-------|---------|-------|----------------|------------------------------------|---------------|
| U-Net (Small) | Custom U-Net (3 down, 2 up) | Adam               | $5 \times 10^{-5}$ | 8     | None    | 60    | 41             | $0.84(1 - \text{SSIM}) + 0.16 L_1$ | ReLU, Sigmoid |



|           |                |      |                    |   |           |    |    |                                    |      |
|-----------|----------------|------|--------------------|---|-----------|----|----|------------------------------------|------|
| DnCNN     | 17-layer DnCNN | Adam | $5 \times 10^{-5}$ | 8 | BatchNorm | 60 | 34 | $0.84(1 - \text{SSIM}) + 0.16 L_1$ | ReLU |
| EDSR (x1) | 8-block EDSR   | Adam | $1 \times 10^{-4}$ | 8 | None      | 60 | 47 | $0.84(1 - \text{SSIM}) + 0.16 L_1$ | ReLU |

Table 3.1: Hyperparameters and configurations of enhancement models.

### 3.4.1 Advanced Preprocessing Pipeline Implementation

#### A. Denoising Network (DnCNN) Implementation

The data processed from the original MRI format through a series of three Neural Networks designed for processing MRI imagery before applying the models during classification as functional parts of the DnCNN Model developed using the residual learning approach eliminates the need to create a model that constructs an output image based on the denoising of an input image. Instead, the DnCNN approach to denoising is to develop a model that identifies and creates a representation of the noise and then subtracts the estimated noise from the original input image to produce a cleaned-up output. The convolutional base of the DnCNN includes an initial convolution and an activation function, and a chain of fifteen convolutional blocks in order to produce a smooth training curve and extract features from

```
# CELL 1: DnCNN (2D) - lightweight version for MRI denoising
import torch
import torch.nn as nn

class DnCNN(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, depth=17, n_feats=64):
        super().__init__()
        layers = []
        # first layer
        layers.append(nn.Conv2d(in_ch, n_feats, 3, padding=1))
        layers.append(nn.ReLU(inplace=True))
        # middle layers
        for _ in range(depth-2):
            layers.append(nn.Conv2d(n_feats, n_feats, 3, padding=1, bias=False))
            layers.append(nn.BatchNorm2d(n_feats))
            layers.append(nn.ReLU(inplace=True))
        # last layer
        layers.append(nn.Conv2d(n_feats, out_ch, 3, padding=1, bias=False))
        # DnCNN learns the noise residual -> output = input - noise
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        residual = self.net(x)
        return torch.clamp(x - residual, 0.0, 1.0)
```

the data and a final output layer with an additional convolution to create the output noise prediction for the original input to be denoised. During the forward pass through the DnCNN model after calculating the residual component, the original input image is then combined with the residual component and clamped to maintain the pixel intensity values in the allowable pixel intensity range to eliminate any artifacts.

## B. Super-Resolution Network (EDSR) Implementation

The EDSR model produces improved clarity by sharpening images, which is particularly beneficial for MRI scans captured at lower-resolution where details can become lost in the background. The architecture uses residual blocks and explicitly avoids using batch normalization so as not to artificially restrict the possible range of pixel values—thus allowing the pixel values to blend together naturally in the data. To provide stability in the training process and to help smooth out fluctuations, we also used a residual scaling factor of 0.1 in our approach to image upscaling. Image upscaling itself is performed using a method called Pixel Shuffle (or sub-pixel convolution), which offers a much better approach

```
# CELL 1: EDSR (Baseline x1 version) adapted for MRI denoising
import torch
import torch.nn as nn

class ResBlock(nn.Module):
    def __init__(self, n_feats, res_scale=0.1):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(n_feats, n_feats, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(n_feats, n_feats, 3, padding=1),
        )
        self.res_scale = res_scale

    def forward(self, x):
        return x + self.block(x) * self.res_scale

class EDSR(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, n_feats=64, n_resblocks=8):
        super().__init__()

        # head
        self.head = nn.Conv2d(in_ch, n_feats, 3, padding=1)

        # body: multiple residual blocks
        body = []
        for _ in range(n_resblocks):
            body.append(ResBlock(n_feats))
        body.append(nn.Conv2d(n_feats, n_feats, 3, padding=1))
        self.body = nn.Sequential(*body)

        # tail
        self.tail = nn.Conv2d(n_feats, out_ch, 3, padding=1)

    def forward(self, x):
        head = self.head(x)
        body = self.body(head)
        out = self.tail(head + body) # residual learning
        return torch.clamp(out, 0.0, 1.0)
```

to image resolution increasing than conventional methods due to the lack of checkerboard artifacts found in those methods.

### **C. Skull Stripping Network (U-Net) Implementation**

To separate brain tissue from the background, we employed a modified U-Net design which is based on the traditional U-Net framework. It includes an encoder (where feature maps are gradually reduced via convolving and pooling layers) and a decoder (where the resolution is restored via up-convoluting layers). Each level of the encoder and decoder communicates through a reduction and a restoration pipeline. With this structure in place, we can capture more intricate features at each stage than if we simply had two separate pipelines (one for reducing features and the other for restoring them).

As such, we can represent our outputs: the final step consists of a  $1 \times 1$  convolution and a sigmoid activation function that produces the binary mask to distinguish pixels of the image representing brain versus non-brain areas.

```

import torch
import torch.nn as nn
class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.net(x)
class UNetSmall(nn.Module):
    def __init__(self, in_ch=1):
        super().__init__()
        self.down1 = DoubleConv(in_ch, 64)
        self.pool = nn.MaxPool2d(2)
        self.down2 = DoubleConv(64, 128)
        self.down3 = DoubleConv(128, 256)
        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.conv2 = DoubleConv(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.conv1 = DoubleConv(128, 64)
        self.out_conv = nn.Conv2d(64, 1, 1)
    def forward(self, x):
        c1 = self.down1(x)
        p1 = self.pool(c1)
        c2 = self.down2(p1)
        p2 = self.pool(c2)
        c3 = self.down3(p2)
        u2 = self.up2(c3)
        u2 = torch.cat([u2, c2], dim=1)
        c4 = self.conv2(u2)
        u1 = self.up1(c4)
        u1 = torch.cat([u1, c1], dim=1)
        c5 = self.conv1(u1)
        out = torch.sigmoid(self.out_conv(c5))
        return out

```

### 3.4.2 Classification Models Implementation

| Model      | Backbone / Architecture      | Input Size    | Optimizer | LR                 | Batch | Epochs (Max) | Early- Stopped |
|------------|------------------------------|---------------|-----------|--------------------|-------|--------------|----------------|
| Simple CNN | 3-layer ConvNet + 2 FC       | 224 × 224 × 3 | Adam      | $1 \times 10^{-4}$ | 16    | 50           | 23             |
| ResNet50   | ResNet50, FC→4               | 224 × 224 × 3 | Adam      | $1 \times 10^{-4}$ | 16    | 50           | 8              |
| VGG16      | VGG16, classifier[6]→4       | 224 × 224 × 3 | Adam      | $1 \times 10^{-4}$ | 16    | 15           | 15             |
| Xception   | Xception (pretrained models) | 224 × 224 × 3 | Adam      | $1 \times 10^{-4}$ | 16    | 50           | 8              |

|                |                              |               |      |                                                               |    |            |    |
|----------------|------------------------------|---------------|------|---------------------------------------------------------------|----|------------|----|
| EfficientNetB4 | EfficientNetB4 + custom head | 380 × 380 × 3 | Adam | 1 × 10 <sup>-3</sup> (head), 1 × 10 <sup>-5</sup> (fine-tune) | 20 | 50 (30+20) | 49 |
|----------------|------------------------------|---------------|------|---------------------------------------------------------------|----|------------|----|

Table 3.2: Hyperparameters and configurations of classification models

Once the MRI images had been preprocessed and enhanced, we fed them into the classification stage. To determine the most effective approach, we trained and evaluated five different neural network architectures side-by-side. The results of this comparative study are presented later in the report. (For the thesis documentation, the recommended code snapshots are shown in the figures referenced below).

### Model 1 – Baseline CNN (From Scratch)

To establish a solid reference point for all subsequent models and enable a direct comparison against future, transfer-learned models, we began by developing a lighter-weight Sequential Model, called a Baseline CNN.

The architecture of the Baseline CNN includes the following components:

- 4 Convolution Layers, with ReLU Activation, Max Pooling layers.
- Each of these components is considered a Convolution Block.
- At the conclusion of the CNN, there are 2 Fully Connected Layers responsible for the Classification of the Images.
- Each Input Image is cropped to the Dimensions of 224 x 224 x 3.
- The chosen Optimizer is Adam, with a Learning Rate of 0.0001.

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32,64,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64,128,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(128*28*28, 256), nn.ReLU(),
            nn.Linear(256, 4)
        )
    def forward(self,x): return self.net(x)

model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
train_acc_list = []
val_acc_list = []
train_loss_list = []
best_acc = 0
patience = 5
no_improve = 0
```

- The Optimizer is configured for a total of 50 Epochs, with the Training being concluded after 23 Epochs because of no further improvements in Validation Performance.

The Baseline CNN trained well, serving as a good baseline from which to measure the performance of all of the Transfer-Learning Architectures to be compared to it.

### Model 2: VGG16 (Transfer Learning)

The VGG16 model was imported with include top=False and all convolutional layers were frozen. This allowed reusing the pretrained filters for edge and texture detection.

- Added Flatten → Dense layers to convert features to class probabilities.
- Suitable for small datasets due to strong generalization.
- Limited by large parameter count.

```
vgg_train_acc = []
vgg_val_acc = []
vgg_train_loss = []

best_acc = 0
patience = 5
no_improve = 0

def train_vgg():
    global best_acc, no_improve

    for epoch in range(15):
        model.train()
        correct = 0
        total = 0
        running_loss = 0

        for imgs, labels in tqdm(train_loader):
            imgs, labels = imgs.to(device), labels.to(device)

            optimizer.zero_grad()
            out = model(imgs)
            loss = criterion(out, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * imgs.size(0)
            _, pred = torch.max(out, 1)
            correct += (pred == labels).sum().item()
            total += labels.size(0)

        train_acc = correct / total
        train_loss = running_loss / total
        val_acc = evaluate(model, val_loader)

        vgg_train_acc.append(train_acc)
        vgg_val_acc.append(val_acc)
        vgg_train_loss.append(train_loss)
```

### Model 3: ResNet50 (Residual Networks)

ResNet50 was chosen due to its skip connections, which prevent vanishing gradients and enable deep network training.

- Used GlobalAveragePooling instead of Flatten to minimize parameters.
- Fast convergence with high accuracy.
- Early stopped at epoch 8.

```

res_train_acc = []
res_val_acc = []
res_train_loss = []

best_acc = 0
patience = 5
no_improve = 0

def train_resnet():
    global best_acc, no_improve

    for epoch in range(50):
        model.train()
        correct = 0
        total = 0
        running_loss = 0

        for imgs, labels in tqdm(train_loader):
            imgs, labels = imgs.to(device), labels.to(device)

            optimizer.zero_grad()
            out = model(imgs)
            loss = criterion(out, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * imgs.size(0)
            _, pred = torch.max(out, 1)
            correct += (pred == labels).sum().item()
            total += labels.size(0)

        train_acc = correct / total
        train_loss = running_loss / total
        val_acc = evaluate(model, val_loader)

        res_train_acc.append(train_acc)
        res_val_acc.append(val_acc)
        res_train_loss.append(train_loss)

```

#### Model 4: Xception (Extreme Inception)

Xception improves efficiency by replacing standard convolutions with Depthwise Separable Convolutions.

- Advantage: It separates the learning of spatial correlations (width/height) from cross-channel correlations (depth), making it computationally lighter than VGG16 while achieving higher accuracy.

```

x_train_acc = []
x_val_acc = []
x_train_loss = []

best_acc = 0
patience = 5
no_improve = 0

def train_xception():
    global best_acc, no_improve

    for epoch in range(50):
        model.train()
        correct = 0
        total = 0
        running_loss = 0

        for imgs, labels in tqdm(train_loader):
            imgs, labels = imgs.to(device), labels.to(device)

            optimizer.zero_grad()
            out = model(imgs)
            loss = criterion(out, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * imgs.size(0)
            _, pred = torch.max(out, 1)
            correct += (pred == labels).sum().item()
            total += labels.size(0)

        train_acc = correct / total
        train_loss = running_loss / total
        val_acc = evaluate(model, val_loader)

        x_train_acc.append(train_acc)
        x_val_acc.append(val_acc)
        x_train_loss.append(train_loss)

```

### Model 5: EfficientNetB4

EfficientNetB4 was chosen as the final classification model due to its strong performance-to-parameter efficiency. It uses **compound scaling**, which uniformly adjusts network width, depth, and resolution, making it effective for high-resolution MRI analysis.

The model receives **380×380 RGB images** from the preprocessing pipeline. Training is performed in two stages to maximise generalisation:



### (a) Data Generators & Class Balancing

```
train_datagen = ImageDataGenerator(  
    preprocessing_function=tf.keras.applications.efficientnet.preprocess_input,  
    rotation_range=15, width_shift_range=0.12, height_shift_range=0.12,  
    shear_range=0.05, zoom_range=0.12, brightness_range=(0.9,1.1),  
    fill_mode='nearest'  
)  
  
test_datagen = ImageDataGenerator(  
    preprocessing_function=tf.keras.applications.efficientnet.preprocess_input  
)  
  
train_generator = train_datagen.flow_from_directory(train_dir,  
                                                    target_size=IMG_SIZE,  
                                                    batch_size=BATCH_SIZE, class_mode='categorical', shuffle=True)  
  
y_train = train_generator.classes  
class_weights = dict(enumerate(class_weight.compute_class_weight(  
    class_weight='balanced', classes=np.unique(y_train), y=y_train)))
```

### (b) Model Architecture & Compilation

```
base_model = EfficientNetB4(include_top=False, input_shape=(*IMG_SIZE,3),  
                            weights='imagenet')  
base_model.trainable = False  
  
x = base_model(inputs, training=False)  
x = layers.GlobalAveragePooling2D()(x)  
x = layers.Dropout(0.45)(x)  
x = layers.Dense(256, activation='relu')(x)  
x = layers.Dropout(0.3)(x)  
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)  
  
model = models.Model(inputs, outputs)  
model.compile(optimizer=optimizers.Adamax(1e-3),  
              loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.08),  
              metrics=['accuracy'])
```

### (c) Two-Stage Training Approach

```

# Stage 1: Train classification head
history = model.fit(train_generator, epochs=int(0.6*EPOCHS),
                    class_weight=class_weights, callbacks=callbacks)

# Stage 2: Fine-tune last 50 layers
for layer in base_model.layers[-50:]: layer.trainable = True
model.compile(optimizer=optimizers.Adam(1e-5), loss=loss, metrics=['accuracy'])
model.fit(train_generator, epochs=EPOCHS, initial_epoch=history.epoch[-1]+1,
          class_weight=class_weights, callbacks=callbacks)

```

### 3.5 Key Challenges and Solutions

While we were working on this project, we encountered some commonplace difficulties related to medical imaging and deep learning. The following list represents most of those encountered as well as the methods used to address them.

1) There were variations in the noise and variances of the MRI scans; although there may be slight differences between images, their disparity may be determined by; the scanner used, the protocol used to conduct the scans, the contrast agents injected, or random noise that can affect the overall quality of an MRI scan. The fluctuating nature and differences of the individual images could be an impediment for the model learning reliable characteristics/features. To lessen this effect, we created an excellent preprocessing pipeline for the model, to include: z-score normalization, clipping of extreme pixel intensities (1st and 99th percentile), denoising by applying the U-Net enhancement model we had previously developed and applied strong data augmentation for (flips, rotations, scaling and elastic deformations, etc.) to ensure that the model is robust to real-world variances in scans.

2) Due to the relatively small number of training scans available, deeper networks, such as ResNet50 and EfficientNetB4, easily memorized data from the training set. Thus, to prevent overfitting to the discipline of the dataset, we employed several common yet proven techniques to reduce the likelihood of overfitting: aggressive augmentation of the training scans, dropout and batch normalization layers, label smoothing, and early stopping once validation loss plateaued.

3. One of the biggest limitations to the model's predictions is class imbalance. As many tumor types were poorly represented in the dataset, which led to the model's predictions of the

majority-tumor types. In order to alleviate this bias, we applied class weighting while training the model (this was especially evident during the EfficientNetB4 phase of training), and we also reported large class representation by using both macro and weighted F1 scores and total accuracy for smaller class representations.

4. Due to GPU memory limitations and the length of time it takes to train the model, we encountered significant challenges when training an EfficientNetB4 model with 380×380 pixel images. However, we were still able to maintain a batch-size of 20, primarily by freezing the majority of the pre-trained layers at the start, and unfroze the last 50 layers to perform fine-tuning of the model. Additionally, we employed mixed precision training with gradient clipping to allow us to conduct all of the model training on one powerful GPU without overloading the memory capacity of the GPU.

5. It can be difficult to deploy large machine learning models if you do not have access to a large cloud-based infrastructure like Google Cloud. To significantly ease the user experience and allow users of all types and expertise to quickly use the system, we have saved both the original Keras & H5 format of the model and built a simple Gradio UI. Anyone can run the demo of Neural Network by executing the command (`python gradio_app.py`), upload their MRI Slices, and receive predictions of your uploaded MRI Slices in seconds without having to perform any complicated installations on their computers.

So addressing and resolving these design issues progressively helped to enable us to create from a prototype capable of providing reliable classification to our users.